Lab 5.2: Adding a Progress Control

In this lab, you will create and implement a progress control.

To see a demonstration of the lab solution, click this icon.
(CD-ROM plays the demonstration, "Lab 5.2 Demonstration.")

Estimated time to complete this lab: **45 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The code that forms the starting point for this lab is located in the folder \Labs\Ch05\Lab5.2.

The solution code for this lab is located in the folder \Labs\Ch05\Lab5.2.

Objectives

After completing this lab, you will be able to:

- Create a progress control.
- Create and control a label for a progress control.
- Display both the control and its label in the context of the status bar.

Prerequisites

There are no prerequisites for this lab.

Exercises

The following exercises provide practice working with the concepts and techniques covered in this chapter:

Exercise 1: Creating a Progress Control

In this exercise, you will add a progress control to the default status bar that MFC AppWizard provides as part of the mainframe class. You will also provide the progress control with a label.

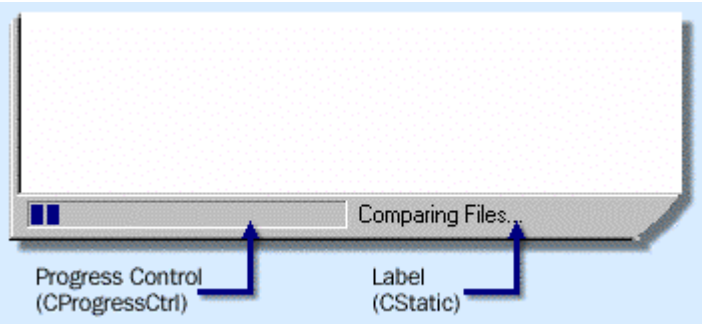
Exercise 2: Implementing the Progress Control

In this exercise, you will provide the supporting code and resources to use the **CProgressStatusBar** class in the ShowDiff application.

159 Exercise 1: Creating a Progress Control

The code that forms the starting point for this exercise is in \Labs\Ch05\Lab5.2\Ex01.

In this exercise, you will add a progress control to the status bar that MFC AppWizard provides as part of the mainframe class. You will also add a label to the progress control as shown in the following illustration.



▶ **Create a new CProgressStatusBar class derived from the generic CWnd class**

☐ 1. Invoke ClassWizard.

☐ 2. Click **Add Class**, click **New,** and add **CProgressStatusBar**. Base it on the generic **CWnd class**.

☐ 3. Click **OK** to create the new class.

☐ 4. Close ClassWizard.

▶ **Edit the generated files to derive CProgressStatusBar from CStatusBar**

☐ 1. Open the file ProgressStatusBar.h.

☐ 2. On the **Edit** menu, click **Replace**.

☐ 3. Replace all instances of **CWnd** with **CStatusBar**.

☐ 4. Save ProgressStatusBar.h.

☐ 5. Repeat this procedure with the file ProgressStatusBar.cpp.

▶ **Add new members to CProgressStatusBar**

CProgressStatusBar needs to track the member controls in the derived class, and the state and position of these member controls. Right-click **CProgressStatusBar** in ClassView and add four protected variables:

☐ 1. Add a member variable for the progress control itself:

```
CProgressCtrl   m_ProgressCtrl
```

☐ 2. Add a member variable for the static control that displays the label:

```
CStatic m_ProgressLabel
```

Notes

☐ 3. **CProgressStatusBar** suppresses the default painting of its **CStatusBar** parent when you display the progress indicator. To track default painting, add a member variable that is set when a progress control is in use:

```
BOOL m_bProgressMode
```

☐ 4. Add a member variable to enable **CProgessStatusBar** to store the width of the progress control to position the label:

```
int m_nProgressCtrlWidth
```

▶ **Create a method to set the width of the progress control**

☐ 1. Right-click **CProgressStatusBar** in ClassView and select **Go to Definition**. Immediately before the class definition, add the following three constants:

First, declare a constant for the default width of the progress control as follows:

```
const int PROGRESS_CTRL_CX = 160;
```

Declare constants for the horizontal and vertical margins and the width of the progress control as follows:

```
const int X_MARGIN = 5; // X value used for margins
                                  // and control spacing
const int Y_MARGIN = 2; // Y value used for margins
                                  // and control spacing
```

☐ 2. Right-click **CProgressStatusBar** in ClassView and add a public member function to set the control width:

```
void SetProgressCtrlWidth(int nWidth = PROGRESS_CTRL_CX)
```

☐ 3. Write the function body to set **m_nProgressCtrlCX** to the passed parameter as follows:

```
void SetProgressCtrlWidth(int nWidth)
{
        m_nProgressCtrlWidth = nWidth;
}
```

☐ 4. Save ProgressStatusBar.cpp.

▶ **Create a method to lay out the progress control and its label**

☐ 1. Right-click **CProgressStatusBar** in ClassView and add a public member function:

```
void RecalcProgressDisplay()
```

☐ 2. Write the code for the **RecalcProgressDisplay** function to get the client rectangle of the status bar as follows:

```
CRect ControlRect;
CRect ClientRect;
GetClientRect(&ClientRect);
ControlRect = ClientRect;
```

You can assume that both the progress control and its labels have been created when this function is called.

Notes

☐ 3. Adjust the rectangle for the progress control so that it reflects the x and y margins, and the progress control width as follows:

```
//  First the Progress bar

ControlRect.left += X_MARGIN;
ControlRect.right = ControlRect.left + m_nProgressCtrlWidth;
ControlRect.top += Y_MARGIN;
ControlRect.bottom -= Y_MARGIN;
```

☐ 4. Position the progress control within this rectangle as follows:

```
m_ProgressCtrl.MoveWindow(ControlRect, FALSE);
```

☐ 5. Adjust the rectangle so that it occupies the space remaining to the right of the progress control, except for the x margins as follows:

```
ControlRect.left = ControlRect.right + X_MARGIN;
ControlRect.right = ClientRect.right - X_MARGIN;
```

☐ 6. Position the label within this rectangle as follows:

```
m_ProgressLabel.MoveWindow(ControlRect, FALSE);
```

☐ 7. Save ProgressStatusBar.cpp.

Notes

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
void CProgressStatusBar::RecalcProgressDisplay()
{
    //  Adjust the postions of the Label and Progress Controls
    //  Place the Label Control to the right of the
    //  Progress Control
    //
    //      [Progress Control] Label Text...

    CRect ControlRect;
    CRect ClientRect;
    GetClientRect(&ClientRect);
    ControlRect = ClientRect;

    //  First the Progress bar

    ControlRect.left += X_MARGIN;
    ControlRect.right = ControlRect.left + m_nProgressCtrlWidth;
    ControlRect.top += Y_MARGIN;
    ControlRect.bottom -= Y_MARGIN;

    m_ProgressCtrl.MoveWindow(ControlRect, FALSE);

    //  Then the text label using the rest of the status
    //  bars client area

    ControlRect.left = ControlRect.right + X_MARGIN;
    ControlRect.right = ClientRect.right - X_MARGIN;

    m_ProgressLabel.MoveWindow(ControlRect, FALSE);
}
```

▶ **Create a method to set the label for the progress control**

☐ 1. Right-click **CProgressStatusBar** in ClassView and add a public member function:

```
void SetProgressLabel(LPCSTR lpszProgressLabel)
```

☐ 2. In the **SetProgressLabel** function, set the text of the static label control to the passed parameter as follows:

```
m_ProgressLabel.SetWindowText(lpszProgressLabel);
```

☐ 3. If the program displays the progress bar at the time of the call, update the display as follows:

```
if(m_bProgressMode)
{
    RecalcProgressDisplay();
    Invalidate();
    UpdateWindow();
}
```

Notes

☐ 4. Save ProgressStatusBar.cpp.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CProgressStatusBar::SetProgressLabel(LPCSTR
                                                    lpszProgressLabel)
{

    m_ProgressLabel.SetWindowText(lpszProgressLabel);

    //  If were currently displaying progress, update
    //  placement of label and progress control

    if(m_bProgressMode)
    {
        RecalcProgressDisplay();
        Invalidate();
        UpdateWindow();
    }
}
```

▶ **Create a method to show or hide the progress control**

☐ 1. Right-click **CProgressStatusBar** in ClassView and add **ShowProgressDisplay** as a public member function:

```
void ShowProgressDisplay(BOOL bShow = TRUE)
```

☐ 2. In the **ShowProgressDisplay** function, set the mode to the passed parameter as follows:

```
m_bProgressMode = bShow;
```

☐ 3. To show the progress control, recalculate the display as follows:

```
if(m_bProgressMode)
{
    RecalcProgressDisplay();
}
```

☐ 4. Show or hide the static and the progress control, as appropriate:

```
m_ProgressLabel.ShowWindow(m_bProgressMode ?    SW_SHOW : SW_HIDE);
m_ProgressCtrl.ShowWindow(m_bProgressMode ?     SW_SHOW : SW_HIDE);
```

☐ 5. Invalidate and update the status bar as follows:

```
Invalidate();
UpdateWindow();
```

☐ 6. Save ProgressStatusBar.cpp.

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CProgressStatusBar::ShowProgressDisplay(BOOL
                                                    bShow)
{
    m_bProgressMode = bShow;
    if(m_bProgressMode)
    {
        RecalcProgressDisplay();
    }
    m_ProgressLabel.ShowWindow(m_bProgressMode ?    SW_SHOW :
                                                        SW_HIDE);
    m_ProgressCtrl.ShowWindow(m_bProgressMode ?     SW_SHOW :
                                                        SW_HIDE);
    Invalidate();
    UpdateWindow();
}
```

▶ **Create a handler for the WM_CREATE message**

☐ 1. Invoke ClassWizard and create a handler for the WM_CREATE message in **CProgressStatusBar**.

☐ 2. Edit this handler to create the progress control with no positions or percentages:

```
if(!m_ProgressCtrl.Create(
                0,                  // Style - Don't Show
                                    //  Position or Percent
                CRect(0,0,0,0), //Initial position
                this,               // Parent ID
                0))                 // Child ID
{
    return -1;
}
```

☐ 3. Create the static label control as follows:

```
if(!m_ProgressLabel.Create(
                NULL,               // Text
                WS_CHILD|SS_LEFT,   // Style
                CRect(0,0,0,0),     // Initial position
                this))              // Parent
{
    return -1;
}
```

☐ 4. Set the font of the static label to the same font as that of the status bar:

```
m_ProgressLabel.SetFont(GetFont());
```

☐ 5. Return 0 to the calling function to indicate success:

```
return 0;
```

☐ 6. Save ProgressStatusBar.cpp.

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
int CProgressStatusBar::OnCreate(LPCREATESTRUCT
                                 lpCreateStruct)
{
    if (CStatusBar::OnCreate(lpCreateStruct) == -1)
        return -1;

    //  Create the Progress Control - we'll calculate its   //  size and
position later - in response to a
    //  ShowProgressDisplay() call.

    if(!m_ProgressCtrl.Create(
                    0,                  // Style - Don't Show
                                        //  Position or Percent
                    CRect(0,0,0,0), //Initial position
                    this,               // Parent ID
                    0))                 // Child ID
    {
        return -1;
    }

    //  Create the Progress Label - we'll calculate its
    //  size and position later - in response to a
    //ShowProgressDisplay() call.

    if(!m_ProgressLabel.Create(
                    NULL,               // Text
                    WS_CHILD|SS_LEFT,   // Style
                    CRect(0,0,0,0), // Initial position
                    this))              // Parent
    {
        return -1;
    }

    //  Use the same font as the Status Bar

    m_ProgressLabel.SetFont(GetFont());

    return 0;
}
```

Notes

▶ Create a handler for the WM_PAINT message

☐ 1. Invoke ClassWizard and create a handler for the WM_PAINT message in **CProgressStatusBar**.

☐ 2. Pass the WM_PAINT message as follows if the progress control does not get displayed:

```
if(!m_bProgressMode)
{
    CStatusBar::OnPaint();
}
```

☐ 3. Save ProgressStatusBar.cpp.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CProgressStatusBar::OnPaint()
{
    //  If were displaying the progress control, then we    //  need to
handle painting of the Status Bar,
    //  otherwise defer to the base class

    if(!m_bProgressMode)
    {
        CStatusBar::OnPaint();
    }
}
```

▶ Update the constructor to set defaults

☐ 1. Set the mode to **FALSE** and the control width to the declared default:

```
m_bProgressMode = FALSE;
m_nProgressCtrlWidth = PROGRESS_CTRL_CX;
```

☐ 2. Save ProgressStatusBar.cpp.

☐ 3. Build your project.

The solution code for this exercise is located in the folder \Labs\Ch05\Lab5.2\Ex01\Solution.

Notes

159 Exercise 2: Implementing the Progress Control

Continue with the files you created in Exercise 1 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch05\Lab5.2\Ex02.

In this exercise, you will provide the supporting code and resources to use the **CProgressStatusBar** class in the ShowDiff application.

▶ **Make the status bar available to other classes**

☐ 1. In the file MainFrm.cpp, include the file ProgressStatusBar.h before the file Splitter.h.

☐ 2. Save MainFrm.cpp.

☐ 3. Open MainFrm.h.

☐ 4. Add a method to return a pointer to the status bar:

```
CProgressStatusBar *GetStatusBar()
    { return &m_wndStatusBar; }
```

☐ 5. Change the status bar type from **CStatusBar** to **CProgressStatusBar**.

☐ 6. Save MainFrm.h.

☐ 7. In ProgressStatusBar.h, add a method to return a pointer to the progress control itself as follows:

```
CProgressCtrl * GetProgressCtrl()
    { return &m_ProgressCtrl; }
```

☐ 8. Save ProgressStatusBar.h.

☐ 9. Include ProgressStatusBar.h before Splitter.h in Diff.cpp, DiffDoc.cpp, and DiffView.cpp.

☐ 10. Save Diff.cpp, DiffDoc.cpp, and DiffView.cpp.

▶ **Use String table editor to add a label string**

☐ 1. Open the String table resource.

☐ 2. Display the properties for the blank line at the end of the String table resource.

☐ 3. Set the **ID** to IDS_COMPARING, and the **Caption** to Comparing files.

☐ 4. Save the String table.

▶ **Set the label of the status bar to the resource string**

☐ 1. In the implementation of **CDiffDoc::RunComparison**, add code as described in the following steps. Place all the code you add before the code that is already there.

☐ 2. In **RunComparison**, before processing the files, get the status bar as follows:

```
CProgressStatusBar *pStatus = CDiffApp::GetApp()->
            GetMainFrame()->GetStatusBar();
```

☐ 3. Add a statement to determine whether pStatus is valid as follows:

```
if (pStatus)
{
```

☐ 4. Declare a **CString** variable as follows:

```
CString Label;
```

Notes

☐ 5. Load the resource string as follows:

```
Label.LoadString(IDS_COMPARING);
```

☐ 6. Set the label of the progress bar as follows:

```
pStatus->SetProgressLabel(Label);
```

▶ **Show the status bar**

- After setting the label, set the status bar to **Progress mode** as follows:

```
pStatus->ShowProgressDisplay(TRUE);
```

▶ **Step the status bar through its range**

Normally, you update a status bar as the result of an action that your program performs. In this procedure, you will simulate that action with a timing loop.

☐ 1. Get a pointer to the progress control as follows:

```
CProgressCtrl *pProgress = pStatus->GetProgressCtrl();
```

☐ 2. Set the range of the progress control to ten steps, starting at 0:

```
pProgress->SetRange(0,10);
```

☐ 3. Display the first step of the range as follows:

```
pProgress->SetStep(1);
```

☐ 4. Do ten steps of the progress control as follows:

```
while (pProgress->StepIt() != 10)
{
    ::Sleep(200);
}
```

▶ **Restore the status bar to its normal behavior**

☐ 1. After you stop the progress control, send the status bar a message to turn off the progress bar as follows:

```
pStatus->ShowProgressDisplay(FALSE);
```

☐ 2. Save DiffDoc.cpp.

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CDiffDoc::RunComparison (LPCSTR lpszFile1, LPCSTR lpszFile2)
{
    CProgressStatusBar *pStatus = CDiffApp::GetApp()->
        GetMainFrame()->GetStatusBar();

    if(pStatus)
    {
        CString Label;
        Label.LoadString(IDS_COMPARING);
        pStatus->SetProgressLabel(Label);

        //  Flip the StatusBar into Progress Mode
        pStatus->ShowProgressDisplay(TRUE);

        //  Simulate some bogus progress
        CProgressCtrl *pProgress = pStatus->GetProgressCtrl();
        if(pProgress)
        {
            pProgress->SetRange(0, 10);
            pProgress->SetStep(1);
            while(pProgress->StepIt() != 10)
            {
                ::Sleep(200);
            }
        }
        pStatus->ShowProgressDisplay(FALSE);
    }

    CMainFrame * pFrame = CDiffApp::GetApp()->GetMainFrame();
    if(pFrame)
    {
        CSplitter * pSplitter = pFrame->GetSplitter();

        if(pSplitter)
        {
            CDiffView * pView;
            pView = (CDiffView *)pSplitter->GetPane(0,0);
            if (pView)
            {
                CFile file(lpszFile1, CFile::modeRead);
                CArchive ar(&file, CArchive::load);
                pView->Serialize(ar);
            }
```

(Code continues on following page.)

Notes

```
            pView = (CDiffView *)pSplitter->GetPane(0,1);
            if (pView)
            {
                CFile file(lpszFile2, CFile::modeRead);
                CArchive ar(&file, CArchive::load);
                pView->Serialize(ar);
            }
            //      Flag as clean so that we won't get
            //      prompted to save
            SetModifiedFlag(FALSE);
        }
    }
}
```

▶ **Build and run your application**

- Click **Open** on the **File** menu and select two files to compare. The status bar will run when comparisons are being simulated between the two files which you have chosen.

The solution code for this exercise is located in the folder \Labs\Ch05\Lab5.2\Ex02\Solution.

Notes